

- Description

This module manages an airport baggage weight check system, ensuring that:

1. **Each passenger's baggage is weighed and checked against the airline's weight limit.**
2. **If the baggage weight exceeds the free allowance, an overweight fee is applied based on excess weight.**
3. **Passengers with baggage exceeding the maximum allowed weight must repack or remove items.**
4. **A log of all baggage weight checks and fees** is maintained for airport tracking and billing.

This system ensures fair enforcement of baggage policies while generating revenue from overweight baggage fees.

- Specification

module AirportBaggageSystem

var baggageWeight, freeWeightLimit, maxWeightLimit, overweightFeePerKg, excessWeight, fee: real

var baggageLog: sequence of string

process checkBaggage(passengerName: string, baggageWeight: real) fee: real

functional scenario form:

baggageWeight > maxWeightLimit && fee = -1, baggageLog = baggageLog
⊕ ("Check-in Denied: " + passengerName), Check-in denied due to exceeding max weight.

||

baggageWeight > freeWeightLimit && excessWeight = baggageWeight – freeWeightLimit, fee = excessWeight * overweightFeePerKg, baggageLog = baggageLog
⊕ ("Overweight Fee: " + passengerName) Overweight fee applied.

||

baggageWeight <= freeWeightLimit && fee = 0, baggageLog = baggageLog
⊕ ("No Fee: " + passengerName), No extra fee.

end process

end module

- Java Code

```
package code;
import java.util.*;
class BaggageCheck {
    private String passengerName;
    private double baggageWeight;
    private final double freeWeightLimit;
    private final double maxWeightLimit;
    private final double overweightFeePerKg;
    private double excessWeight;
```

```

    private double fee;
    public BaggageCheck(String passengerName, double baggageWeight, double freeWeightLimit, double maxWeightLimit, double overweightFeePerKg)
    {
        this.passengerName = passengerName;
        this.baggageWeight = baggageWeight;
        this.freeWeightLimit = freeWeightLimit;
        this.maxWeightLimit = maxWeightLimit;
        this.overweightFeePerKg = overweightFeePerKg;
        this.excessWeight = calculateExcessWeight();
        this.fee = calculateFee();
    }
    private double calculateExcessWeight() {
        if (baggageWeight > freeWeightLimit) {
            return baggageWeight - freeWeightLimit;
        } else {
            return 0;
        }
    }
    private double calculateFee() {
        if (baggageWeight > maxWeightLimit) {
            return -1;
        } else if (excessWeight > 0) {
            return excessWeight * overweightFeePerKg;
        } else {
            return 0;
        }
    }
    public boolean isOverweight() {
        return fee > 0;
    }
    public boolean isRejected() {
        return fee == -1;
    }
    public String getBaggageDetails() {
        if (isRejected()) {
            return String.format("Passenger: %s | Baggage Weight: %.1f kg | EXCEEDS MAX LIMIT! Check-in Denied.",
                passengerName, baggageWeight);
        } else if (isOverweight()) {
            return String.format("Passenger: %s | Baggage Weight: %.1f kg | Excess: %.1f kg | Fee: $%.2f",
                passengerName, baggageWeight, excessWeight, fee);
        } else {

```

```

        return String.format("Passenger: %s | Baggage Weight: %.1f
kg | Within Limit, No Extra Fee",
        passengerName, baggageWeight);
    }
}
}
class AirportBaggageSystem {
    private List<String> baggageLog;
    public AirportBaggageSystem() {
        this.baggageLog = new ArrayList<>();
    }
    public void checkBaggage(String passengerName, double weight) {
        BaggageCheck check = new BaggageCheck(passengerName, weight, 20
.0, 32.0, 10.0);
        System.out.println(check.getBaggageDetails());
        if (!check.isRejected()) {
            baggageLog.add(check.getBaggageDetails());
        }
    }
    public void printBaggageLog() {
        System.out.println("\nBaggage Check Log:");
        for (String log : baggageLog) {
            System.out.println(log);
        }
    }
}
public class c21_AirportSystem {
    public static void main(String[] args) {
        AirportBaggageSystem system = new AirportBaggageSystem();
        // testcase-VT:
        //     system.checkBaggage("Alice", 18.0);
        //     system.checkBaggage("Bob", 23.0);
        //     system.checkBaggage("Charlie", 35.0);
        //     system.checkBaggage("David", 30.0);

        // testcase-FT:
        //     system.checkBaggage("", -1.7976931348623157E308);
        //     system.checkBaggage("??", 5.350177392258598E291);
        //     system.checkBaggage("??mmm", 1.508684118918264E306);
        //     system.checkBaggage("fhao0--", 1.5071135814397465E306);

        // testcase-Z3:
        //     system.checkBaggage("Alice", 17);
        //     system.checkBaggage("Bob", 19);
    }
}

```

```
//      system.checkBaggage("Charlie", 17);
//      system.checkBaggage("David", 19);

// testcase-UVT:
system.checkBaggage("Alice", 18);
system.checkBaggage("Bob", 23);
system.checkBaggage("Charlie", 17);
system.checkBaggage("David", 30);
system.printBaggageLog();
}
}
```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	2.1 %	18	824	842
▼ src	2.1 %	18	824	842
▼ code	2.1 %	18	824	842
▼ c21_AirportSystem.java	100.0 %	18	0	18
> AirportBaggageSystem	100.0 %	4	0	4
> BaggageCheck	100.0 %	14	0	14
> c21_AirportSystem		0	0	0

3. Vibration testing instruction coverage: 98.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.3 %	218	16,495	16,713
▼ src	1.3 %	218	16,495	16,713
▼ code	1.3 %	218	16,495	16,713
▼ c21_AirportSystem.java	98.6 %	218	3	221
> c21_AirportSystem	88.5 %	23	3	26
> AirportBaggageSystem	100.0 %	50	0	50
> BaggageCheck	100.0 %	145	0	145

4. Fuzz testing branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.8 %	15	827	842
▼ src	1.8 %	15	827	842
▼ code	1.8 %	15	827	842
▼ c21_AirportSystem.java	83.3 %	15	3	18
> BaggageCheck	78.6 %	11	3	14
> AirportBaggageSystem	100.0 %	4	0	4
> c21_AirportSystem		0	0	0

5. Fuzz testing instruction coverage: 82.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.1 %	182	16,531	16,713
▼ src	1.1 %	182	16,531	16,713
▼ code	1.1 %	182	16,531	16,713
▼ c21_AirportSystem.java	82.4 %	182	39	221
> BaggageCheck	75.2 %	109	36	145
> c21_AirportSystem	88.5 %	23	3	26
> AirportBaggageSystem	100.0 %	50	0	50

6. Z3 branch coverage: 55.6%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	830	840
▼ src	1.2 %	10	830	840
▼ code	1.2 %	10	830	840
▼ c21_AirportSystem.java	55.6 %	10	8	18
> BaggageCheck	50.0 %	7	7	14
> AirportBaggageSystem	75.0 %	3	1	4
> c21_AirportSystem		0	0	0

7. Z3 instruction coverage: 70.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.7 %	156	21,939	22,095
▼ src	0.7 %	156	21,939	22,095
▼ code	0.7 %	156	21,939	22,095
▼ c21_AirportSystem.java	70.6 %	156	65	221
> BaggageCheck	57.2 %	83	62	145
> c21_AirportSystem	88.5 %	23	3	26
> AirportBaggageSystem	100.0 %	50	0	50

8. Unimproved V-method branch coverage: 77.8%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.7 %	14	826	840
▼ src	1.7 %	14	826	840
▼ code	1.7 %	14	826	840
▼ c21_AirportSystem.java	77.8 %	14	4	18
> AirportBaggageSystem	75.0 %	3	1	4
> BaggageCheck	78.6 %	11	3	14
> c21_AirportSystem		0	0	0

9. Unimproved V-method instruction coverage: 89.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	198	21,897	22,095
▼ src	0.9 %	198	21,897	22,095
▼ code	0.9 %	198	21,897	22,095
▼ c21_AirportSystem.java	89.6 %	198	23	221
> BaggageCheck	86.2 %	125	20	145
> c21_AirportSystem	88.5 %	23	3	26
> AirportBaggageSystem	100.0 %	50	0	50